Detecting Malware Activities over HTTPS Connection using Record Sequences

Presenter : Jiwon Yang

(jwyang@gist.ac.kr)

Network and Security Lab.

School of Electrical Engineering and Computer Science (EECS)

Gwangju Institute of Science and Technology (GIST)

Contents

01 Background

02 Motivation

03 Proposed Solution

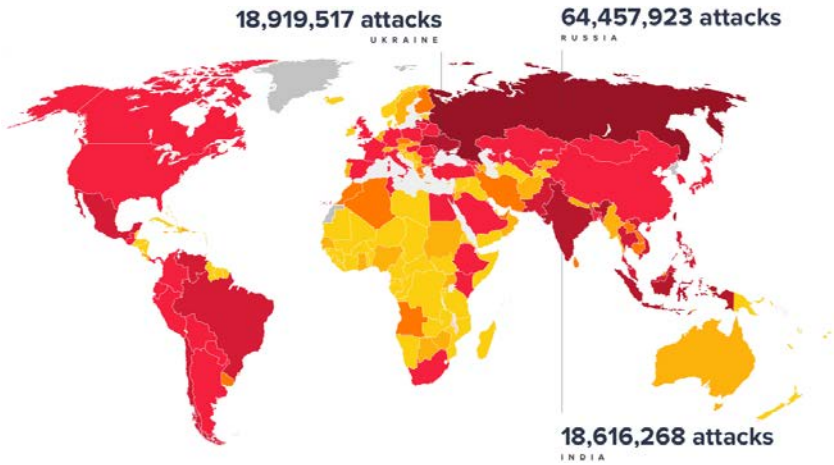
04 Experiment

05 Conclusion

Live cyber threats worldwide

* A major cyber attack has affected many security experts across the world

- ✓ In May 2017, WannaCry emerged and spread like wildfire across the Internet, encrypting user's files and demanding cryptocurrency to unlock
- ✓ WannaCry had earned more than $143,000 through bitcoin payments



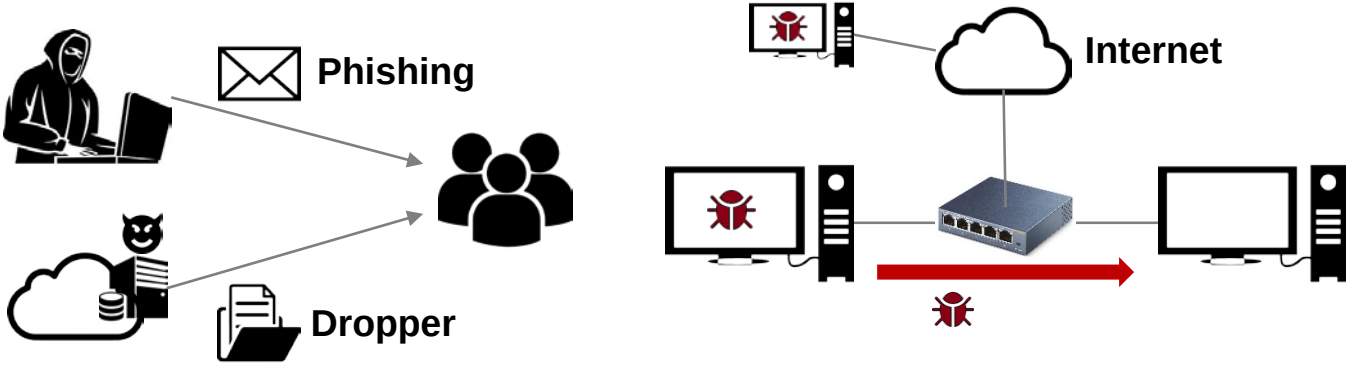
▲Global WannaCry cyber attack hits 150 countries in 2017

What's behind that incident?

* A malware that propagates through a network **without** any user involvement

✓ WannaCry is a dangerous combination of two malicious components:

self-propagating worm module + backdoor module



▲Traditional malware infection route (left) and WannCry infection route (right)

It can be the tip of the iceberg,
Numerous attack vectors are evolving more swiftly

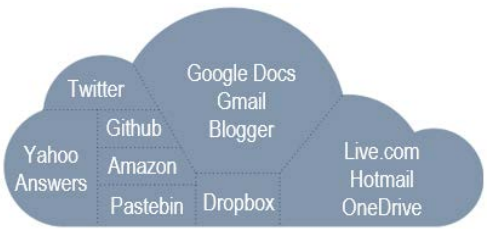
Evasion skills of emerging malware

* They're out there: Defenders should prepare to face advanced strategies:

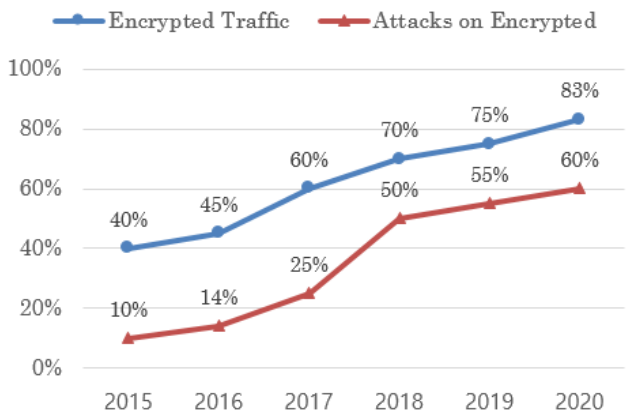
① Exploit legitimate Internet services (Google, Dropbox, etc.) or tools (Windows Powershell, Microsoft Word)

② Bring multiple campaigns together through a single callback

③ Embrace the encryption techniques



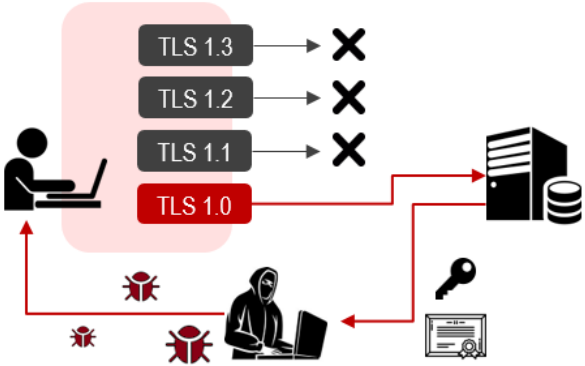
▲Examples of legitimate services abused by malware for C&C



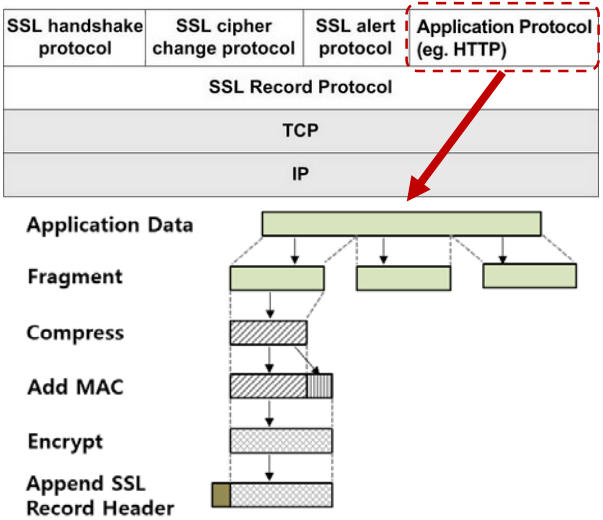
▲Increase in volume of malicious binaries leveraging encrypted network

Sneaky Malware in Secure Socket Layer

- ✓ Secure Socket Layer (SSL) is the most preferred cryptographic protocol
  - : Easy to obtain certificate & exploit vulnerabilities in earlier versions
- ✓ Attackers tend to negotiate low-level security parameters during the handshaking process
- ✓ Malware vendors hide their contents inside of Application Data Record



▲Downgrade the protocol version to exploit vulnerability



▲Attacker forces the victim to downgrade the protocol version

Idea to combat SSL malware

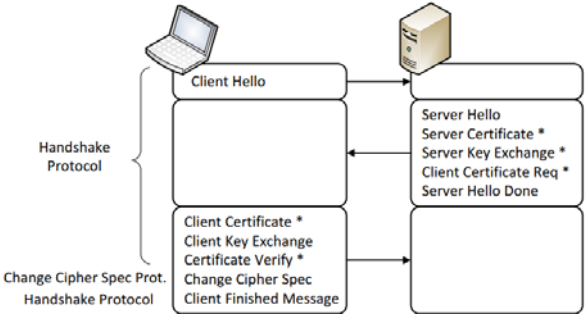
* Not see the forest for the trees

Attackers tend to negotiate **low-level security parameters** during the **handshaking process**

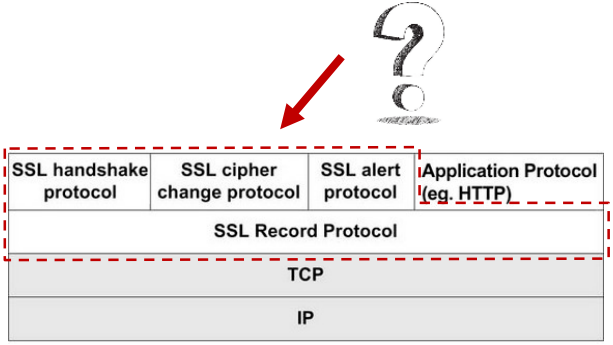
- How much difference is there? (between normal/malware traffic)

Malware vendors **hide their contents** inside of **Application Data Record**

- What about the others?



▲What stage can be optional?



▲Are they plaintext?

Related Works

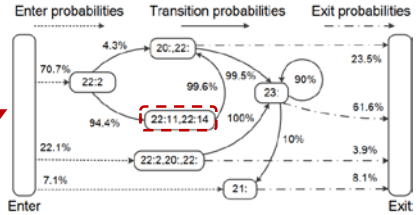
* **We can expect unique patterns from the record exchange process**

✓ The stochastic Markov chains reflect the unique behaviors of SSL session according to the application nature

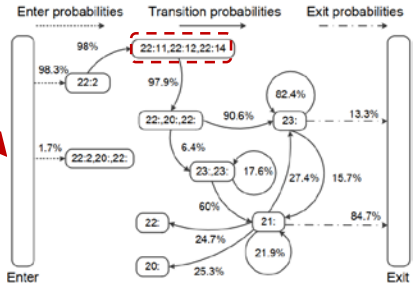
Decimal Code	Protocol Type
20	Change Cipher Spec
21	Alert
22	Handshake
23	Application Data

Decimal Code	Handshake Message Type
0	Hello Request
1	Client Hello
2	Server Hello
11	Certificate
12	Server Key Exchange
13	Certificate Request
14	Server Hello Done
15	Certificate Verify
16	Client Key Exchange
20	Finished

▲**SSL message types and their corresponding decimal codes**



◀ **Markov diagram for PayPal**



◀ **Markov diagram for Twitter**

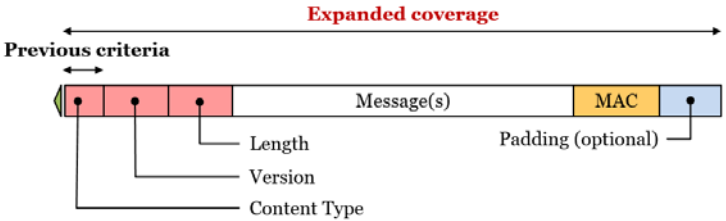
M. Korczynski and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," INFOCOM, IEEE, 2014

How to improve such a idea?

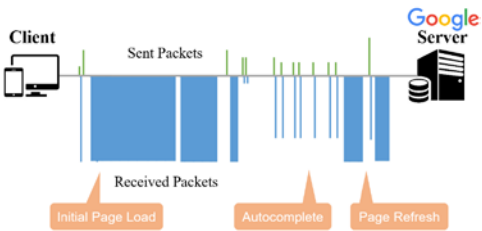
Our Strategies for Improvements

* Improve such ideas to provide more visibility into threat landscape :

- ✓ Expand the feature-space from record type to whole record field data
  - ▶ It requires **parsing & removal of encrypted fields**



- ✓ Automatically figure out the fluctuating characteristics (e.g., periodicity gradualness, and singularity) in record sequence
  - ▶ It requires **advanced pattern recognition techniques**

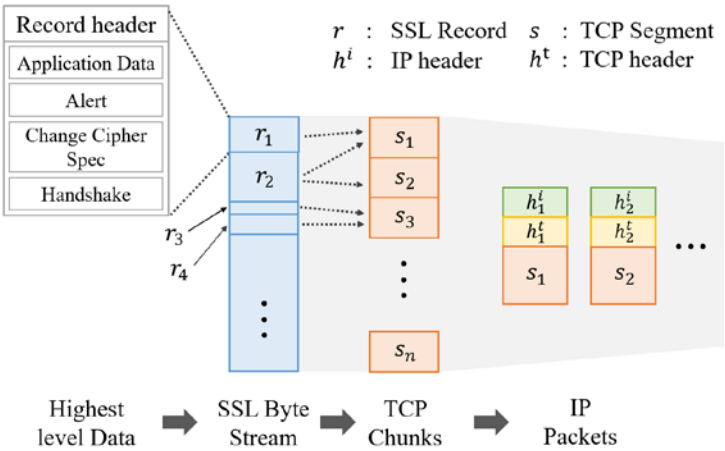


▲Like a packet frequency pattern, anything else in a record?

Pre-Processing

✓ We parsed the SSL flow:

- rearrange the packet sequence based on the TCP seq number
- abandon the payloads not associated with SSL record
- append record fragments with valid syntax to original record frame



▲Conversion process of record messages to a series of IP packet

Algorithm 1 Extract records from a TCP packet

```
procedure PARSERECORD(pakcet P, flow-table T)
    S ←T[P]                                  ▷ associated TCP stream
    if previous segment not captured then
        S.queue.push(P)                      ▷ delay the parsing order
        return
    else if P is retransmitted packet then
        return
    end if
    p ← P.payload                            ▷ payload pointer
    if P is neither keep-alive nor window-probe then
        while p is not at end of payload do
            record header validation (first 5 bytes)
            l ← current record-length
            extract p-th ~ (p + l − 1)-th byte values
            p ← p + l
        end while
    end if
    P′ ←S.queue.front                        ▷ packet with lowest seq
    if P′.seq == S.nextSEQ then
        ParseRecord(P′, T)                   ▷ parse the reserved packet
    end if
    return
end procedure
```

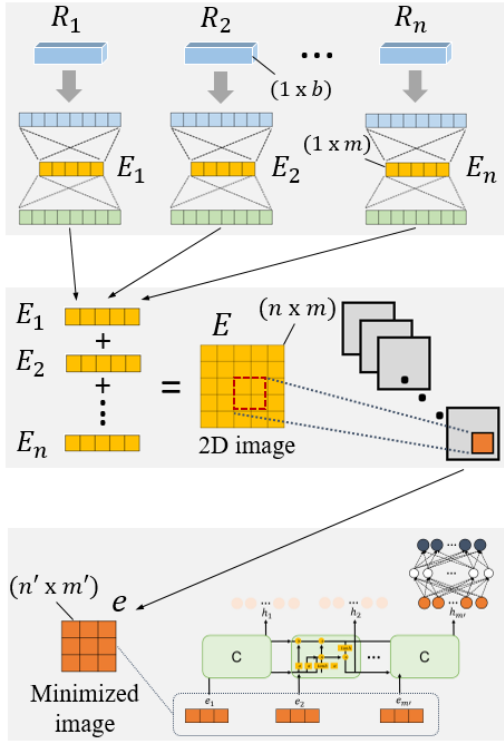
▲SSL Record parsing algorithm

Classification Method

① arrange first 'n' records and reduce the dimension of each record $\mathbb{R}^b \longrightarrow \mathbb{R}^m$

② combine the extracted vectors vertically, and apply another autoencoding to the 2D image E, $\mathbb{R}^{(n \times m)} \longrightarrow \mathbb{R}^{(n' \times m')}$

③ fed the minimized image e to the classifier that produces the output, 0 : normal, 1 : malicious



Record feature extraction
(1D auto encoding)

Flow compression
(2D auto encoding)

Classification
(CNN, LSTM)

Dataset & Metrics

* Dataset is composed of:

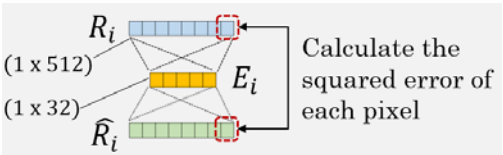
- Normal : 113,539 flows from GIST
- Malware : 37,153 flows from public data source (Palo Alto Networks, Inc.)

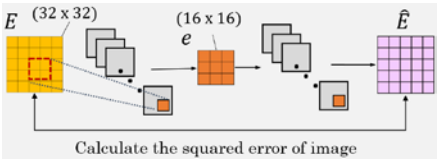
* The evaluation metrics for detector's performance:

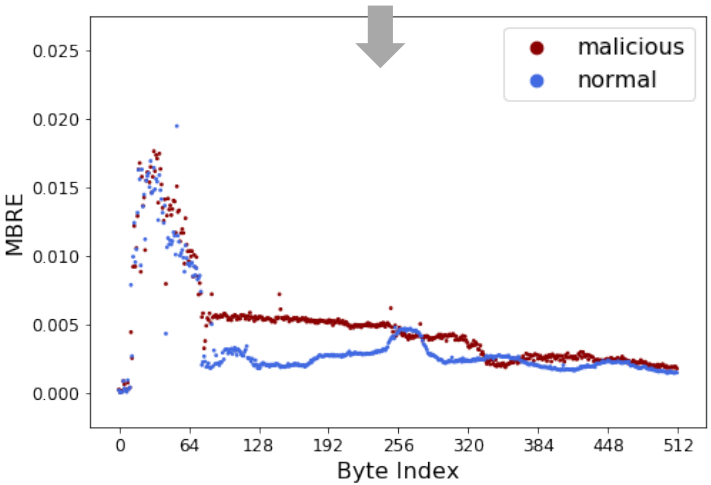
- $$\text{Detection Rate (DR)} = \frac{\text{\#correctly identified malicious flow samples}}{\text{\#malicious flow samples}}$$
- $$\text{False Alarm Rate (FAR)} = \frac{\text{\#misclassified normal flow samples}}{\text{\#normal flow samples}}$$

Result #1 : Feature Reconstruction

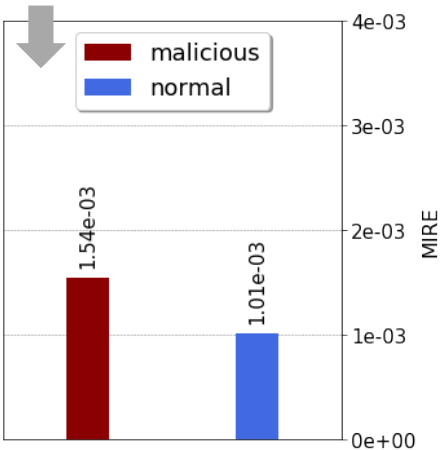
* **Malrecords deny being ordinary as their reconstruction errors suggest**
  - ✓ Even trained with both normal/malware traffic, malicious flow samples show **higher reconstruction** error in both 1st & 2nd encoding phases







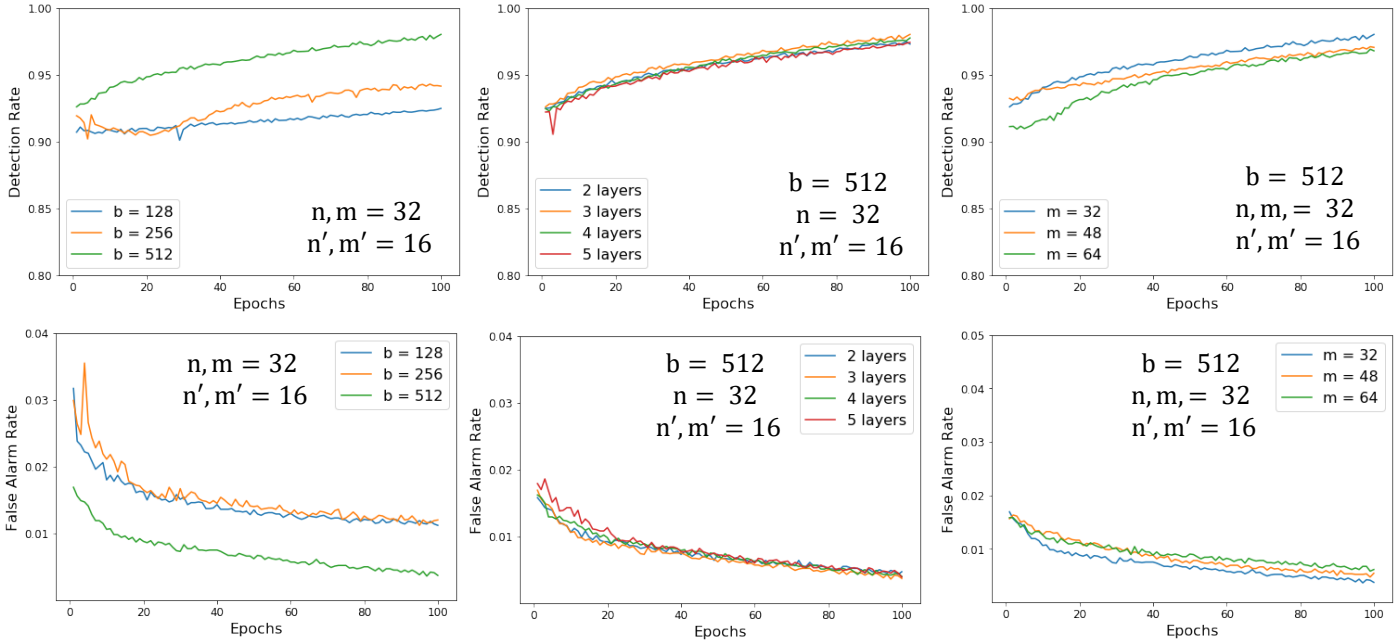
▲Mean reconstruction error for each record byte value



▲Mean reconstruction error for encoded image

Result #2 : Classification Performance

- ✓ The **size of input record (*b*)** matters
- ✓ But, the number of layers in classifier & the size of encoded vector (m) do not influence the performance much



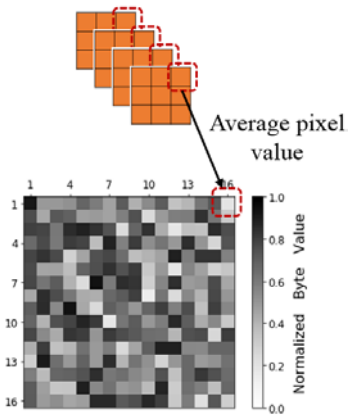
▲**Performance of LSTM classifiers with different parameter setting**

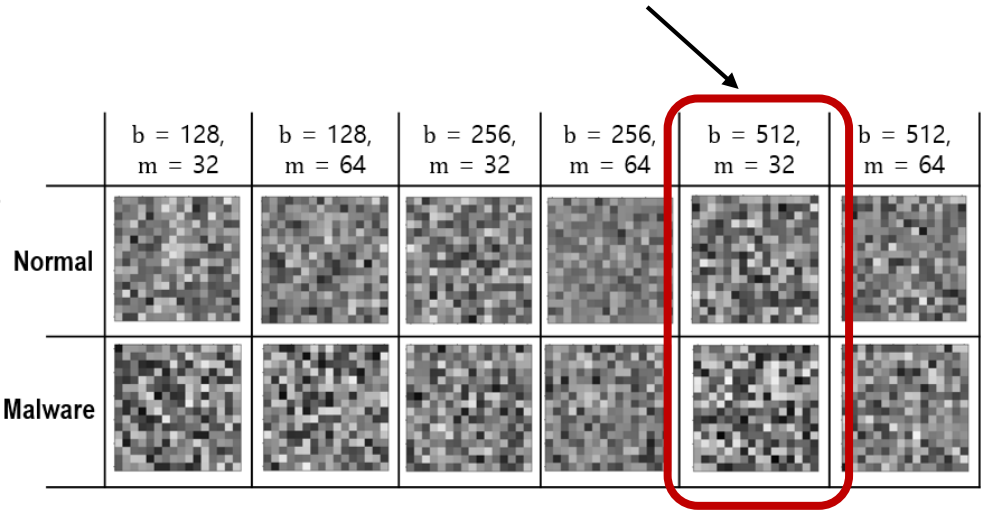
Analysis #1 : Feature Map of C&C Callback

* **From the sight of the classifier, the texture of malware images is apparent**
  - ✓ Depending on the parameters, the position of darker spots varies, but we can feel subtle dissimilarity between two images for every case

Where the difference came from?

Minimized images

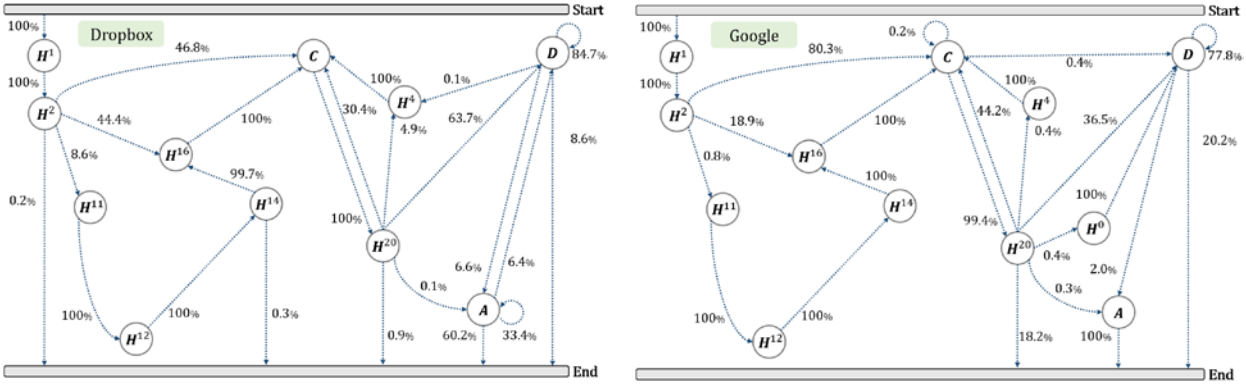




▲**Encoded Image of normal/malicious traffic sample (**$n = 32$ **,** $n' = m' = 16$**)**

Analysis #2 : Footprint of Benign Flow

- ✓ The differences in handshaking mainly come from their preferred configuration (security parameters, negotiation method, etc.)
- ✓ Record exchange process after the handshaking reflects the flow features like the session duration or the total record size



Handshake Message:

H^0 : Hello Request	H^1 : Clinet Hello
H^2 : Server Hello	H^4 : New Session Ticket
H^{11} : Certificate	H^{12} : Server Key Exchange
H^{14} : Server Hello Done	H^{16} : Client Key Exchange
H^{20} : Finished	H^{22} : Certificate Status

Non - Handshake Message:

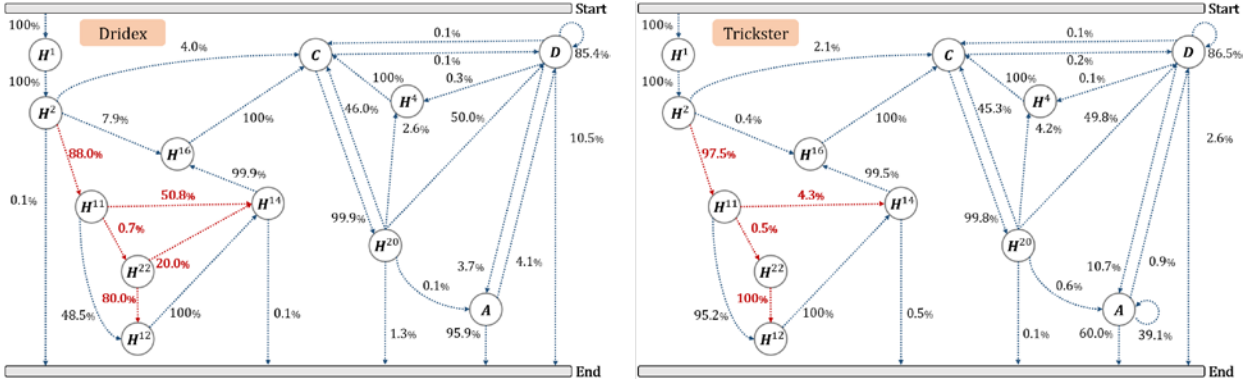
A : Alert	C : Change Cipher Spec
D : Application Data	

▲**Footprints of two different benign applications**

Analysis #3 : Footprint of C&C Callback

* A dark spot for security professionals : SSL certification

- ✓ The C&C server shows high certificate exchange frequencies to legitimate services (red arrows)
- ✓ Some malware binaries contain valid or stolen certificates to skip several handshaking phases



Handshake Message:

H^0 : Hello Request	H^1 : Clinet Hello
H^2 : Server Hello	H^4 : New Session Ticket
H^{11} : Certificate	H^{12} : Server Key Exchange
H^{14} : Server Hello Done	H^{16} : Client Key Exchange
H^{20} : Finished	H^{22} : Certificate Status

Non - Handshake Message:

A : Alert	C : Change Cipher Spec
D : Application Data	

▲Footprints of two different malware variants

Conclusion & Future Works

* **The proposed malware detection utilizes a sequence of SSL record**
  - ✓ The results demonstrate the presence of malware-associated patterns in a series of record
  - ✓ The proposed method is capable of accurately identifying such patterns

* **Still, newly emerging threats are out-of-radar**
  - ✓ How to update malware pattern continuously?
  - ✓ What should be done if we encounter a high false alarm rate?

Thank You.